

CrickSense — V1 requirements

Test-cricket analytics dashboard and AI search — starting with Pakistan, built to expand to other countries.

1. Vision

A public web app that lets anyone explore a country's Test cricket performance and player weaknesses through a dashboard and an AI-powered search bar. V1 focuses entirely on Pakistan to prove the product out; a countries menu is built in from day one so other countries (Australia, Sri Lanka, India, England, etc.) can be switched on later without a rebuild. Fun and open for fans, but built with credible, structured analysis so the same foundation can later support a cricket board or team management use case.

2. V1 scope

In scope

- Format: Test cricket only. T20 and ODI shown in the nav bar as "Soon".
- Country: Pakistan only, selected by default in a countries menu (hamburger). Other countries listed but marked "Soon".
- Opponents: all teams Pakistan has played, matches from 2021 onwards.
- Data granularity: ball-by-ball — runs, dismissal type, bowler, batsman, phase of innings. No line/length or pitch-map data.
- Interface: dashboard-first, with an AI search bar that interprets intent and jumps to the right view.
- Player weakness analysis for both batting and bowling.
- ICC Test ranking shown per player.
- One-line AI-generated tactical suggestion per player.
- PDF export of whatever is currently on screen.
- Post-match analysis only — no live score/live match support.

Out of scope for V1 (future phases)

- Other countries' data (Australia, Sri Lanka, India, England, etc.) — added after the Pakistan V1 is validated.
- Live match data or live scoring.
- T20 and ODI formats.
- Line/length, pitch maps, release-point analysis — needs paid data sources.
- Board-specific private data or video footage.

3. Data sources

Data	Source	Notes
Ball-by-ball match data	Cricsheet.org (free)	Every ball: batsman, bowler, runs, extras, wicket type, fielder. Covers all Pakistan Tests 2021+.
ICC Test rankings	ICC official site (scraped/manual)	No free API. Updated roughly weekly — refreshed on a schedule, not real-time.
Player photos	Public sources / manual upload	Licensing to be checked before public launch.

Limitation to flag: without line/length data, "weakness" is defined at the level we can measure — dismissal type, bowler type (pace/spin/swing), and phase of innings. E.g. "caught behind vs pace in overs 1–10 in 60% of dismissals", not "weak outside off on a good length". This can be upgraded later if paid data is added.

4. Infrastructure and tech stack

Chosen to keep V1 cost at zero, with a clear upgrade path if usage grows.

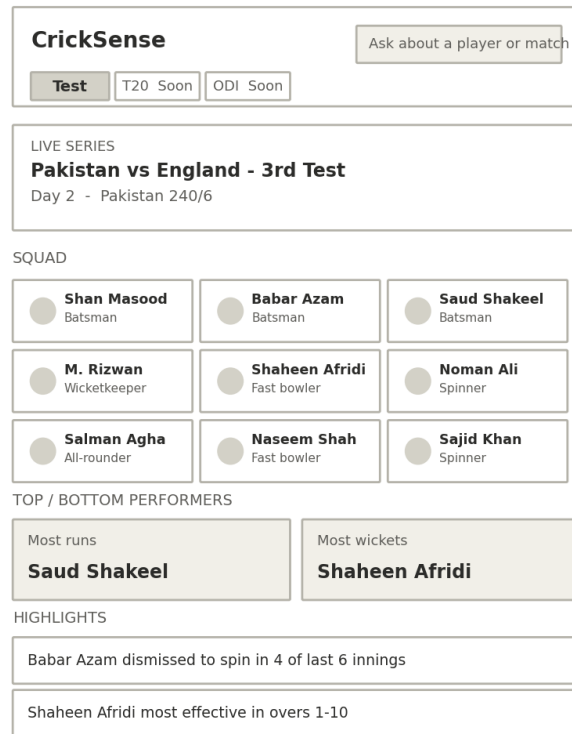
Layer	Choice	Why
Frontend + backend	Next.js, one repo	API routes double as the backend — one codebase, one deploy.
Hosting	Vercel (free tier)	Native fit for Next.js. Also runs the scheduled ICC scraper job for free.
Database	Neon (serverless Postgres, free tier)	Free, plugs into Vercel/Next.js with minimal setup.
Data pipeline	Python scripts	Download and parse Cricsheet data, load into Neon.
AI layer	Groq (free API), Gemini for testing	Groq is primary — free, fast, hosted open-source models (Llama etc.). Gemini used in testing to compare output quality before committing. No self-hosting for V1.
ICC rankings	Scheduled scraper (Vercel cron)	No free official API exists, so this is scraped on a schedule, not real-time.

Everything above stays within free tiers for V1. If usage grows, Neon and Vercel both have straightforward paid upgrade paths without needing to switch tools. Self-hosting an open-source model is a future option, not needed for V1 — it requires paid GPU hosting even at low usage, unlike Groq's free API.

5. Screens (lo-fi mockups)

5.1 Dashboard homepage

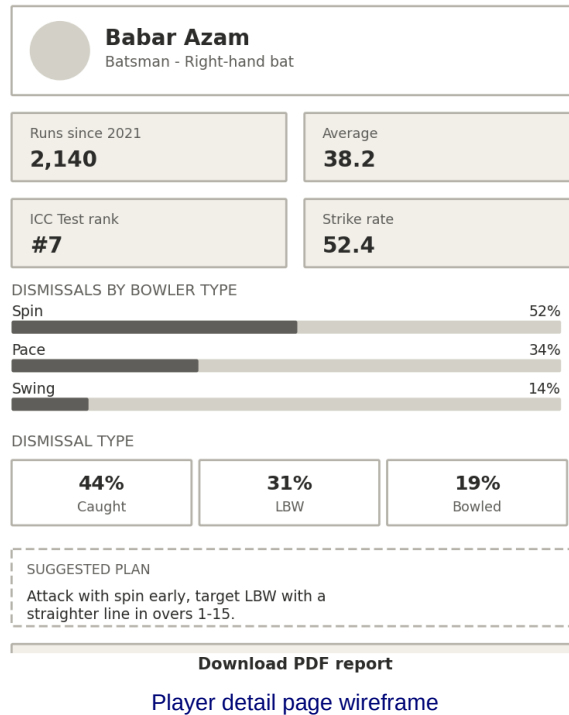
Default landing view. Nav bar (Test active, T20/ODI marked Soon) with search bar. Below: live series scoreboard, squad grid (photo, name, role — 3 per row, full squad shown), top/bottom performer cards, and auto-generated highlight cards.



Homepage wireframe — default landing view

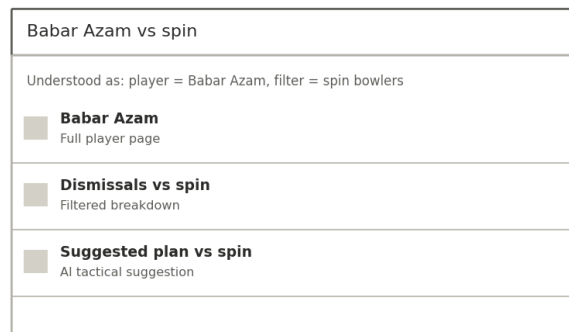
5.2 Player detail page

Opens when a player is selected from the squad grid or via search. Shows header (photo, name, role), key stat cards including ICC rank, dismissals-by-bowler-type breakdown, dismissal-type breakdown, an AI-generated suggested plan, and a PDF export button.



5.3 Search / ask bar behavior

Typing a free-text question (e.g. "Babar Azam vs spin") shows: what the AI understood (player + filter), then three quick options — full player page, filtered breakdown, or straight to the tactical suggestion. Tapping one drives the dashboard to that view.



5.4 Countries menu

A hamburger menu opens a countries list. Pakistan is selected by default for V1; every other country is listed but marked "Soon", so the door is visibly open for expansion without implying it works yet.

COUNTRIES

● **Pakistan**

Australia

Soon

Sri Lanka

Soon

India

Soon

England

Soon

South Africa

Soon

Countries menu wireframe

6. V1 build plan — atomic steps

Phase 0 — infrastructure setup

- 1 Create Next.js project, single repo for frontend and backend.
- 2 Connect repo to Vercel, confirm a blank deploy works.
- 3 Create Neon Postgres project and connect it to the Next.js app.
- 4 Create a Groq account and confirm a test API call works.

Phase 1 — data pipeline

- 1 Download Cricsheet Test match files for Pakistan, 2021 to present.
- 2 Parse match files into flat ball-by-ball records.
- 3 Design database schema: matches, innings, deliveries, players.
- 4 Load parsed data into the database.
- 5 Write aggregation query: runs, average, strike rate per player.
- 6 Write aggregation query: dismissal type percentage per player.
- 7 Write aggregation query: dismissals by bowler type (pace/spin/swing) per player.
- 8 Write aggregation query: performance by innings phase (e.g. overs 1–10, 11–40, 40+).
- 9 Source ICC Test rankings and attach to player records.
- 10 Set up a refresh job to re-run the pipeline as new match data appears.

Phase 2 — backend / API

- 1 Set up backend project skeleton.
- 2 Endpoint: current series and scoreboard.
- 3 Endpoint: squad list (photo, name, role).
- 4 Endpoint: player detail (stats, dismissal breakdowns, ICC rank).
- 5 Endpoint: top/bottom performers.
- 6 Endpoint: highlight/insight cards (rule-based to start).
- 7 Endpoint: parse free-text search into structured intent (player, filter, view).
- 8 Connect the intent-parsing endpoint to Groq's free API.
- 9 Test the same intent-parsing prompts against Gemini and compare output quality.
- 10 Endpoint: generate the one-line AI tactical suggestion per player.

Phase 3 — frontend

- 1 Set up frontend project skeleton.
- 2 Build nav bar (Test active, T20/ODI marked Soon).
- 3 Build countries hamburger menu (Pakistan default, others marked Soon).

- 4 Build search bar with dropdown (intent line + 3 jump options).
- 5 Build homepage: live series card.
- 6 Build homepage: squad grid, 3 columns, full squad.
- 7 Build homepage: top/bottom performer cards.
- 8 Build homepage: highlight cards.
- 9 Build player page: header (photo, name, role).
- 10 Build player page: stat cards (runs, average, ICC rank, strike rate).
- 11 Build player page: dismissals-by-bowler-type bars.
- 12 Build player page: dismissal-type breakdown cards.
- 13 Build player page: AI suggested-plan box.
- 14 Build PDF export button.
- 15 Wire search bar to backend intent endpoint and jump logic.

Phase 4 — PDF export

- 1 Decide per-screen PDF template (homepage report vs player report).
- 2 Implement "download as PDF" for whatever is on screen.

Phase 5 — QA and polish

- 1 Test with real 2021+ data across the full current squad.
- 2 Validate dismissal percentages against known scorecards.
- 3 Get informal feedback from a few cricket fans.
- 4 Fix data gaps — retired players, name mismatches across sources.

Phase 6 — launch

- 1 Deploy publicly (infrastructure to be decided next).
- 2 Add a feedback/suggestion box for users.
- 3 Monitor usage and collect input for phase 2 (T20/ODI, richer ball data).

7. Next step

Infrastructure is now decided (Next.js + Vercel + Neon + Groq). Next: share this document with the design tool to produce updated mockups reflecting the CrickSense name and the countries menu, then begin Phase 0.